Exam & Quiz

Generate a complete deployable project.

🎯 SYSTEM OVERVIEW

Build a production-ready **Exam & Quiz Management SaaS** using:

* Next.js 16 (App Router)
* TypeScript
* Prisma ORM
* Neon PostgreSQL (Pooler)
* NextAuth (Admin authentication)
* Tailwind + ShadCN UI
* Recharts (dashboard analytics)
* Zod validation
* SWR polling (5 sec refresh)
* Excel import (.xlsx)
* Vercel deployment ready

🔐 ENVIRONMENT VARIABLES

Use this Neon Pooler connection:

```

DATABASE_URL=postgresql://neondb_owner:npg_9Nyz5dvwUAul@ep-muddy-hill-a1a0gl2w-pooler.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require&pgbouncer=true&connection_limit=20
```

```

NEXTAUTH_URL=http://localhost:3000

NEXTAUTH_SECRET=BWWITySQrQZ3BPMfwlqhVVbEpknIPyb12insIkW0hmo=
```

Must be production-ready for Vercel.

🏗️ CORE ARCHITECTURE

1⃣ Quiz Template System

A Quiz acts as a template.

A Quiz can have multiple QuizBatches.

Each batch:

* Has independent duration
* Has independent start/end time
* Has active/inactive toggle
* Stores its own responses

* Has its own leaderboard

🗄 DATABASE DESIGN (Full Prisma Schema Required)

Include:

* Admin
* Quiz
* QuizBatch
* Question
* Option
* Response
* Answer
* LeaderboardSnapshot
* AuditLog

Add required indexes for performance.

Requirements:

* Unique constraint on (batchId, email)
* Index on batchId
* Index on totalScore
* Store IP address
* Store device fingerprint hash
* Store startedAt
* Store submittedAt

* Question must have marks field
* Option must support correct flag

🏆 LEADERBOARD SYSTEM

Scoring Logic

* Each question has marks.
* Single correct: full marks.
* Multiple correct:
  * Full marks if all correct selected.
  * Optional partial scoring.
* Text answer:
  * Manual grading supported.

Ranking Rules

Sort by:

1. totalScore DESC
2. submittedAt ASC

Assign rank dynamically.

Dashboard

Admin must see:

* Top 10
* Full leaderboard
* Pagination
* CSV export
* Toggle leaderboard visibility
* Auto refresh every 5 seconds

Cache leaderboard for 5 seconds.

🛡 ANTI-CHEATING SYSTEM

1 IP Lock

* Capture IP from headers.
* Store in Response.
* Configurable:

  * Strict mode: one submission per IP.
* Flag suspicious IP clusters.

2 Device Lock

Generate device fingerprint using:

* userAgent

* timezone

* screen resolution

Hash and store deviceHash.

Block:

* Same device multiple emails (configurable).

3 Suspicious Behavior Detection

Flag if:

* Submission < 20% of duration

* Multiple attempts same IP

* Multiple attempts same device

Log in AuditLog table.

🎓 EXAM MODE (STRICT MODE)

If batch.examMode = true:

Rules:

* No back navigation.

* Disable browser back button.
* Lock question navigation.
* Full-screen enforcement (optional).
* Detect tab switching:

  * Show warning
  * Log event
* Auto-submit on:

  * Timer expiry
  * Browser close (best effort)
  * Inactivity timeout

Backend must verify:

* startedAt + duration
* Reject late submission

⏱ TIMER SYSTEM

* Duration stored in QuizBatch.
* When participant clicks "Start Exam":

  * Create Response record
  * Save startedAt
* Timer calculated server-side.
* Frontend shows countdown.

* Auto-submit when expired.

Server must revalidate expiration.

📊 REALTIME DASHBOARD

Use SWR polling every 5 seconds.

Show:

* Total participants
* Live submissions count
* Submission timeline chart
* Question distribution
* Suspicious activity alerts
* Leaderboard preview

No WebSockets.

📥 EXCEL IMPORT FEATURE

Admin can upload .xlsx file.

Excel Format:

| question | type | marks | optionA | optionB | optionC | optionD | correctAnswers |

| -------- | ---- | ----- | ------- | ------- | ------- | ------- | -------------- |

Type values:

* SINGLE
* MULTIPLE
* TEXT

CorrectAnswers:

* For SINGLE: A
* For MULTIPLE: A,C
* For TEXT: expected answer (optional)

Requirements:

* Parse using xlsx library.
* Validate with Zod.
* Show preview before saving.
* Bulk insert with transaction.
* Rollback on failure.
* Error report if format invalid.

📂 ROUTING STRUCTURE

```
```

```
app/
  (public)/
    quiz/[batchSlug]/
      page.tsx
      start/page.tsx
      exam/page.tsx
      submit/page.tsx

  (admin)/
    admin/
      dashboard/
      quizzes/
      quizzes/create/
      quizzes/[id]/
      batches/[id]/results/
      import/

  api/
    quizzes/
    batches/
    responses/
    leaderboard/
    import/
```

🔐 ADMIN PANEL FEATURES

* Create quiz template
* Add questions manually
* Import questions from Excel
* Create multiple batches
* Toggle active/inactive
* Enable/disable:

  * Exam mode
  * IP lock
  * Device lock
  * Leaderboard visibility
* View suspicious logs
* Export:

  * Responses CSV
  * Leaderboard CSV

🔎 SECURITY

* All forms validated with Zod
* Server-side validation mandatory
* Rate limit submission API
* Sanitize text answers
* Protect admin routes via NextAuth middleware
* Environment variable validation

🚀 DEPLOYMENT INSTRUCTIONS

Include:

* prisma generate
* prisma migrate deploy
* Seed script for admin
* Production build configuration
* Vercel deployment steps
* Neon pooler configuration
* .env.example file

📈 PERFORMANCE OPTIMIZATION

* Use indexes
* Use pagination
* Use Prisma include to prevent N+1
* Cache leaderboard 5 sec
* Limit heavy aggregation queries
* Use incremental static regeneration for public pages

SEED DATA

Generate:

* Default admin user

* Sample quiz

* Sample batch

* Sample questions

🎨 UI REQUIREMENTS

* Modern SaaS design

* Dark/light mode

* Responsive

* Loading skeletons

* Toast notifications

* Clear error states

* Professional dashboard layout

🎁 OUTPUT REQUIREMENT

Generate:

* Full project structure

* All source files

* Prisma schema

* Middleware

* API routes

* React components
* Leaderboard logic
* Anti-cheat logic
* Excel import logic
* Exam mode logic
* Deployment guide
* README.md

Project must be ready to:

```
npm install
npx prisma generate
npx prisma migrate deploy
npm run build
vercel deploy
```

No pseudo-code.